

Bokeh for Time Series Analysis



****Introduction to Bokeh**** Bokeh is an interactive visualization library for Python that targets modern web browsers for presentation. Unlike Matplotlib, which is primarily designed for static plots, Bokeh excels at creating interactive plots and dashboards. It can handle large datasets and streaming data, making it suitable for real-time applications.

Key Features of Bokeh:

- **Interactivity:** Built-in support for zooming, panning, hovering, and other interactive tools.
- **Web-Focused:** Generates HTML and JavaScript, making it easy to embed plots in web pages.
- **High Performance:** Can handle large datasets efficiently.
- **Versatility:** Supports a wide range of plot types (lines, bars, scatter plots, etc.).

Documentation:

For comprehensive documentation, please refer to the official Bokeh website: <https://docs.bokeh.org/en/latest/>

Lab Exercise:

Your task is to recreate the time series analysis lab we previously conducted using Pandas, Matplotlib, and Seaborn, but this time, utilize the Bokeh library for visualization. This will involve:

1. Loading and preprocessing the "AirPassengersDates.csv" dataset.
2. Creating interactive Bokeh plots for:
 - Time series line plots
 - Bar plots of aggregated data
 - Visualizing mean and standard deviation
 - Outlier detection
 - Resampling (upsampling and downsampling)
 - Lag analysis
 - Autocorrelation

Pay close attention to Bokeh's features for interactivity (tools, hover effects) and its handling of data sources. Aim to replicate the insights and visualizations from the previous lab while leveraging Bokeh's strengths.

Good luck!

```
In [3]: import pandas as pd
import numpy as np
from bokeh.plotting import figure, output_notebook, show
from bokeh.models import HoverTool

output_notebook()

passenger_df = pd.read_csv("datasets/AirPassengersDates.csv")
passenger_df["Date"] = pd.to_datetime(passenger_df["Date"])

print(passenger_df.head())
print(passenger_df.info())
```



BokehJS 3.8.2 successfully loaded.

```

      Date  #Passengers
0 1949-01-01         112
1 1949-02-01         118
2 1949-03-01         132
3 1949-04-01         129
4 1949-05-01         121
<class 'pandas.DataFrame'>
RangeIndex: 144 entries, 0 to 143
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Date        144 non-null    datetime64[us]
1   #Passengers  144 non-null    int64
dtypes: datetime64[us](1), int64(1)
memory usage: 2.4 KB
None

```

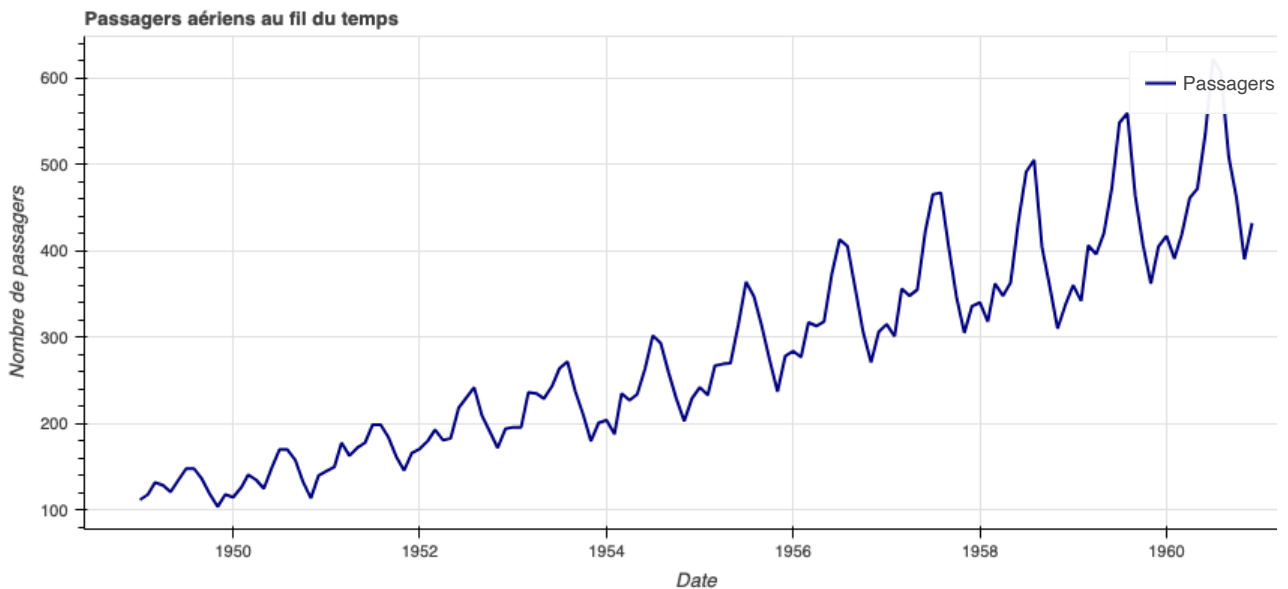
```

In [4]: # Graphique temporel
p1 = figure(
    title="Passagers aériens au fil du temps",
    x_axis_label="Date",
    y_axis_label="Nombre de passagers",
    x_axis_type="datetime",
    width=900,
    height=400
)

p1.line(passenger_df["Date"], passenger_df["#Passengers"],
        line_width=2, color="navy", legend_label="Passagers")

hover1 = HoverTool(tooltips=[("Date", "@x{%F}"), ("Passagers", "@y")],
                  formatters={"x": "datetime"})
p1.add_tools(hover1)
p1.legend.click_policy = "hide"
show(p1)

```



```

In [5]: # Agrégation par mois
passengers_per_month = passenger_df.groupby(
    passenger_df["Date"].dt.month
)["#Passengers"].sum().reset_index()
passengers_per_month.columns = ["Mois", "Total_Passagers"]

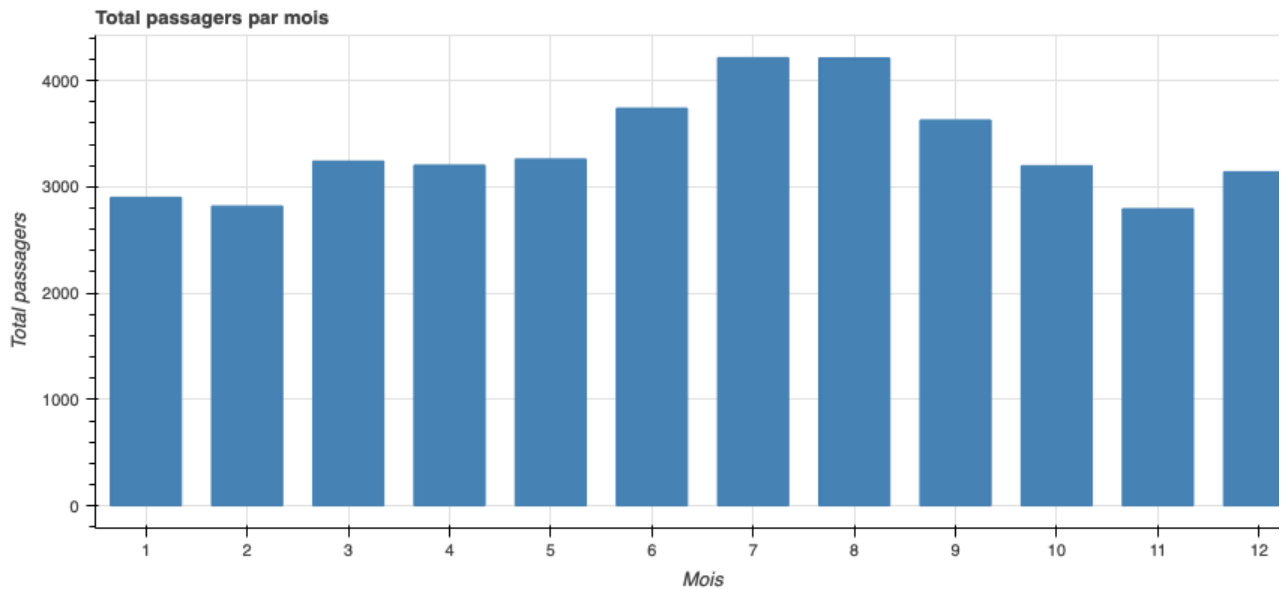
p2 = figure(
    title="Total passagers par mois",
    x_axis_label="Mois",
    y_axis_label="Total passagers",
    width=900,
    height=400,
    x_range=[str(int(m)) for m in passengers_per_month["Mois"]]
)

```

```

p2.vbar(
    x=[str(int(m)) for m in passengers_per_month["Mois"]],
    top=passengers_per_month["Total_Passagers"],
    width=0.7,
    color="steelblue"
)
show(p2)

```



```

In [6]: # Moyenne et écart-type
mean_passengers = passenger_df["#Passengers"].mean()
std_passengers = passenger_df["#Passengers"].std()

p3 = figure(
    title="Passagers avec bandes moyenne et écart-type",
    x_axis_label="Date",
    y_axis_label="Nombre de passagers",
    x_axis_type="datetime",
    width=900,
    height=400
)

p3.line(passenger_df["Date"], passenger_df["#Passengers"],
        line_width=2, color="navy", legend_label="Passagers")

p3.line(passenger_df["Date"],
        [mean_passengers] * len(passenger_df),
        line_dash="dashed", color="red", line_width=2,
        legend_label=f"Moyenne: {mean_passengers:.0f}")

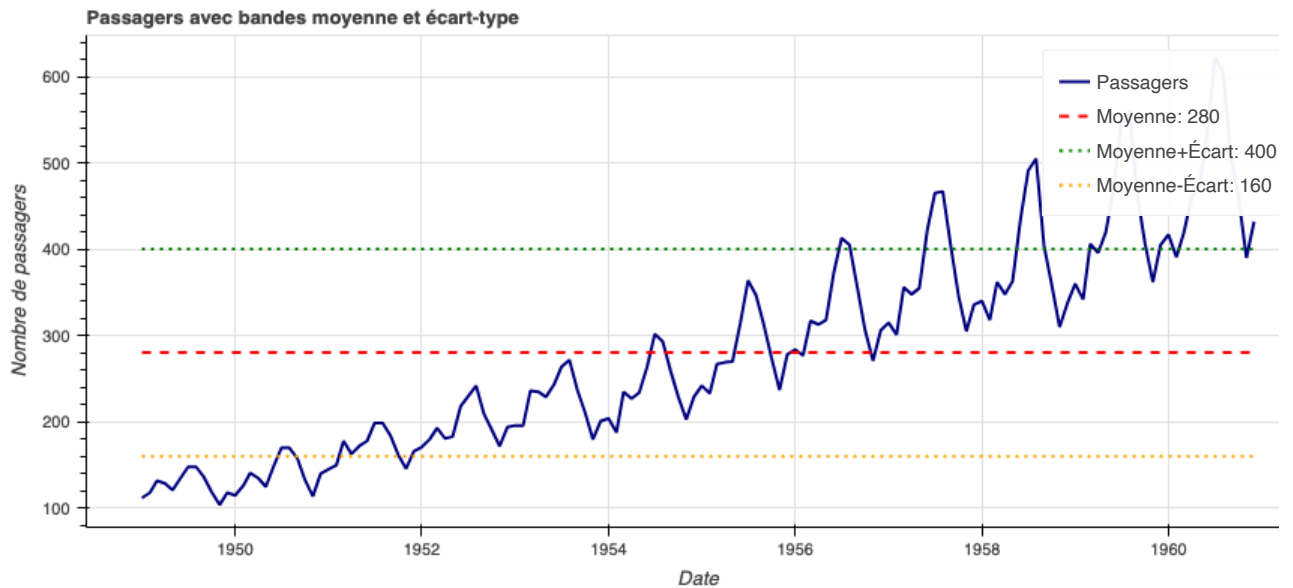
p3.line(passenger_df["Date"],
        [mean_passengers + std_passengers] * len(passenger_df),
        line_dash="dotted", color="green", line_width=2,
        legend_label=f"Moyenne+Écart: {mean_passengers + std_passengers:.0f}")

p3.line(passenger_df["Date"],
        [mean_passengers - std_passengers] * len(passenger_df),
        line_dash="dotted", color="orange", line_width=2,
        legend_label=f"Moyenne-Écart: {mean_passengers - std_passengers:.0f}")

p3.legend.click_policy = "hide"
show(p3)

print(f"Moyenne: {mean_passengers:.2f}")
print(f"Écart-type: {std_passengers:.2f}")

```



Moyenne: 280.30
Écart-type: 119.97

```
In [7]: # Détection valeurs aberrantes
passenger_df["z_score"] = np.abs((passenger_df["#Passengers"] - mean_passengers) / std_passengers)
outliers = passenger_df[passenger_df["z_score"] > 3]

p4 = figure(
    title="Détection valeurs aberrantes (Z-score > 3)",
    x_axis_label="Date",
    y_axis_label="Nombre de passagers",
    x_axis_type="datetime",
    width=900,
    height=400
)

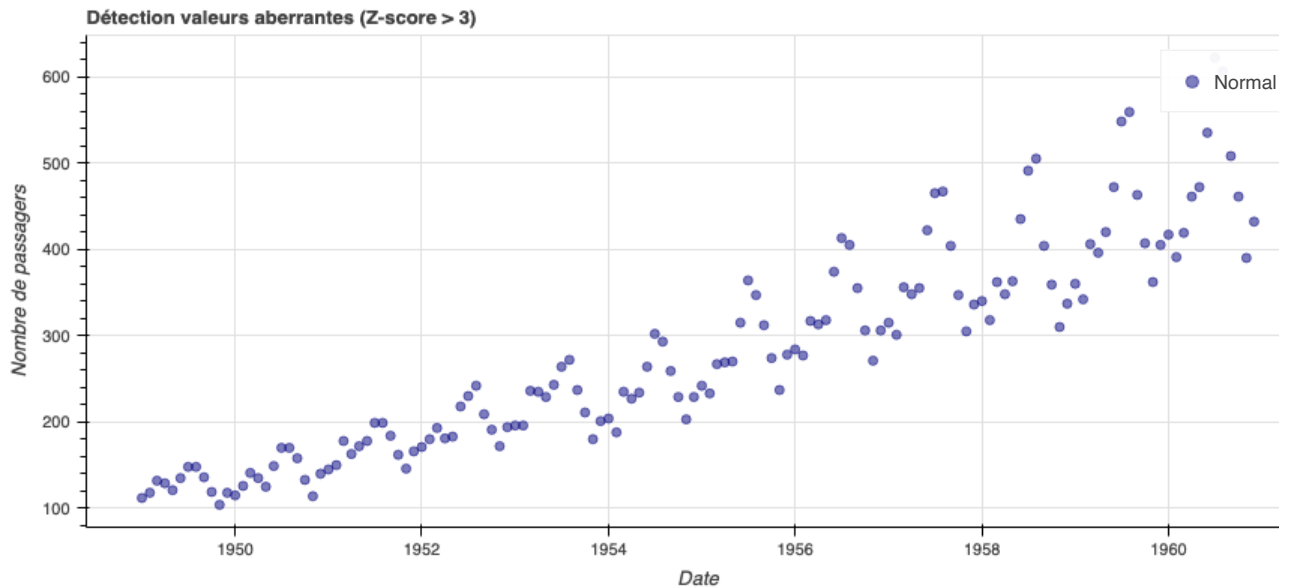
p4.circle(passenger_df["Date"], passenger_df["#Passengers"],
          size=6, color="navy", alpha=0.5, legend_label="Normal")

if len(outliers) > 0:
    p4.circle(outliers["Date"], outliers["#Passengers"],
              size=10, color="red", legend_label="Anomalies")

p4.legend.click_policy = "hide"
show(p4)

print(f"Anomalies détectées: {len(outliers)}")
print(outliers)
```

BokehDeprecationWarning: 'circle() method with size value' was deprecated in Bokeh 3.4.0 and will be removed, use 'scatter(size=...) instead' instead.



Anomalies détectées: 0
 Empty DataFrame
 Columns: [Date, #Passengers, z_score]
 Index: []

```
In [8]: # Rééchantillonnage - Moyenne annuelle
passenger_resampled = passenger_df.set_index("Date")["#Passengers"]
yearly_avg = passenger_resampled.resample("YE").mean()

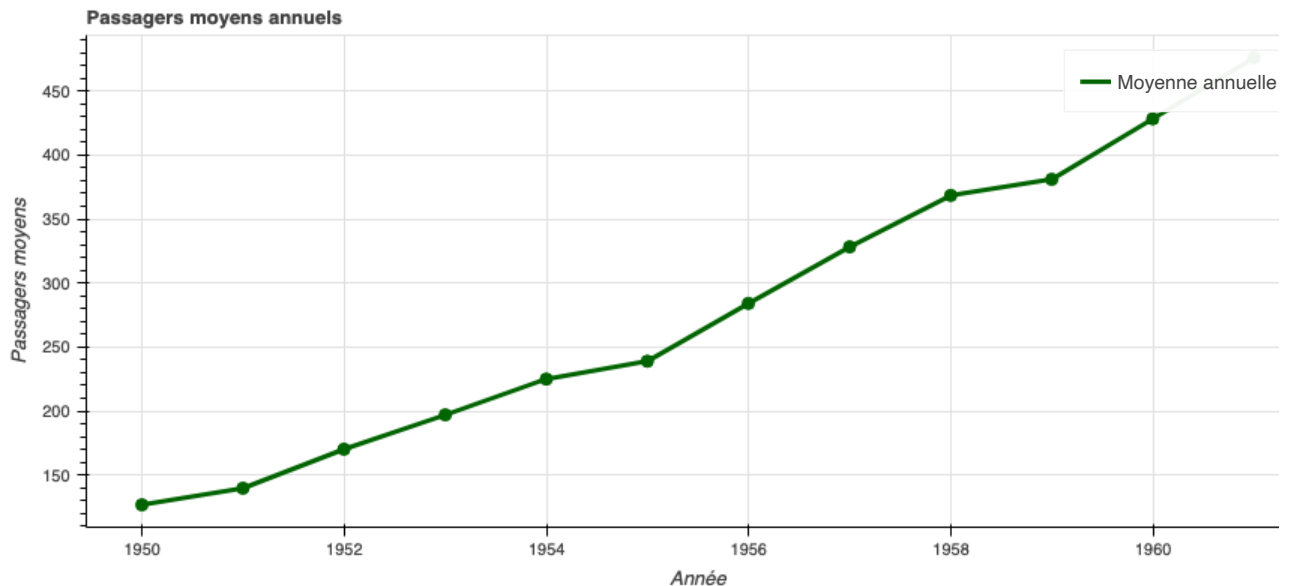
p5 = figure(
    title="Passagers moyens annuels",
    x_axis_label="Année",
    y_axis_label="Passagers moyens",
    x_axis_type="datetime",
    width=900,
    height=400
)

p5.line(yearly_avg.index, yearly_avg.values,
        line_width=3, color="darkgreen", legend_label="Moyenne annuelle")
p5.circle(yearly_avg.index, yearly_avg.values,
          size=8, color="darkgreen")

p5.legend.click_policy = "hide"
show(p5)

print(yearly_avg)
```

BokehDeprecationWarning: 'circle()' method with size value' was deprecated in Bokeh 3.4.0 and will be removed, use 'scatter(size=...) instead' instead.



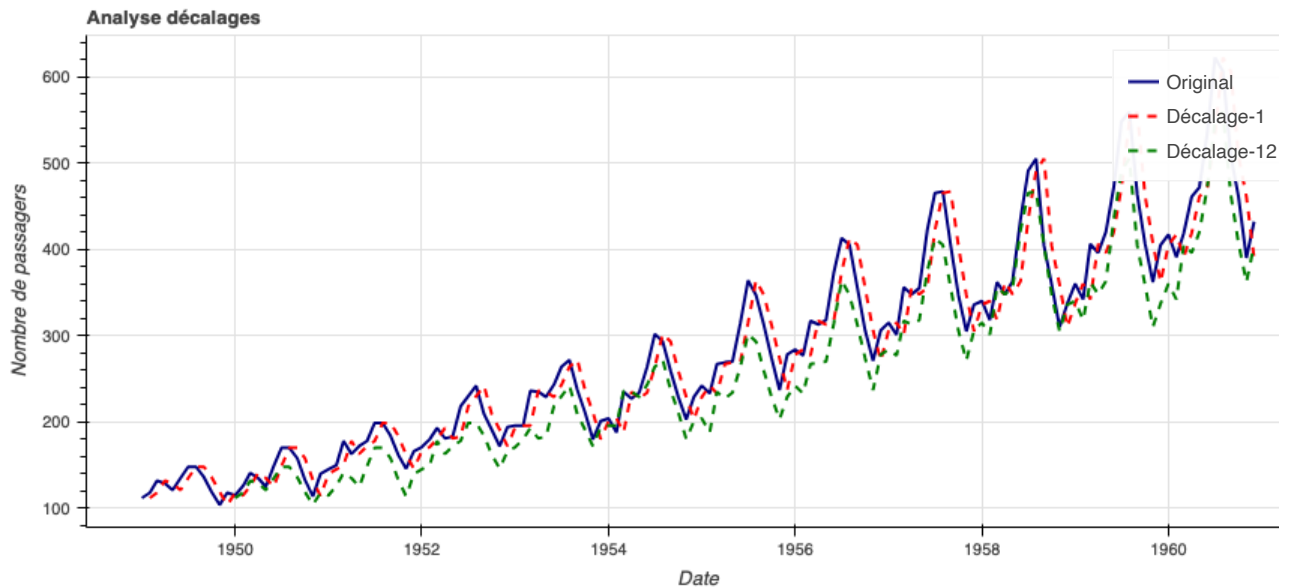
```
Date
1949-12-31    126.666667
1950-12-31    139.666667
1951-12-31    170.166667
1952-12-31    197.000000
1953-12-31    225.000000
1954-12-31    238.916667
1955-12-31    284.000000
1956-12-31    328.250000
1957-12-31    368.416667
1958-12-31    381.000000
1959-12-31    428.333333
1960-12-31    476.166667
Freq: YE-DEC, Name: #Passengers, dtype: float64
```

```
In [9]: # Analyse décalages
passenger_df["decalage_1"] = passenger_df["#Passengers"].shift(1)
passenger_df["decalage_12"] = passenger_df["#Passengers"].shift(12)

p6 = figure(
    title="Analyse décalages",
    x_axis_label="Date",
    y_axis_label="Nombre de passagers",
    x_axis_type="datetime",
    width=900,
    height=400
)

p6.line(passenger_df["Date"], passenger_df["#Passengers"],
        line_width=2, color="navy", legend_label="Original")
p6.line(passenger_df["Date"], passenger_df["decalage_1"],
        line_width=2, color="red", line_dash="dashed", legend_label="Décalage-1")
p6.line(passenger_df["Date"], passenger_df["decalage_12"],
        line_width=2, color="green", line_dash="dashed", legend_label="Décalage-12")

p6.legend.click_policy = "hide"
show(p6)
```



```
In [10]: # Autocorrélation
from pandas.plotting import autocorrelation_plot
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(12, 6))
autocorrelation_plot(passenger_df["#Passengers"], ax=ax)
plt.title("Graphique autocorrélation")
plt.xlabel("Décalage")
plt.ylabel("Autocorrélation")
plt.grid(True)
plt.show()
```

